

**Presented by :** Sothang Meitei

**Presentation on the paper :** A new image encryption based on bit replacing, chaos and DNA coding techniques Sura F. Yousif<sup>1</sup> & Ali J. Abboud<sup>2</sup> & Raad S. Alhumaima

# *Table of content*

- Introduction and objectives
- Background and prerequisites
- Encryption and decryption algorithm and flowchart
- Bit replacing of the image file
- Confusion and diffusion using chaotic systems
- Application of the DNA coding algorithm
- Analysis of the performance of the proposed encryption algorithm
- Conclusion

# *Introduction and objective*

Image files are needed to be protected using secure encryption techniques that can then be successfully decrypted on the receiver's end with least amount of information lost and high efficiency.

Here is the presentation on the paper

A new image encryption based on bit replacing, chaos and DNA coding techniques

Sura F. Yousif<sup>1</sup> & Ali J. Abboud<sup>2</sup> & Raad S. Alhumaima

For this a strong and efficient cryptographic system is needed

Following are the things that are required from such a system

1. This system must protect the image file from external agents that aims to detect and extract as much information of the encrypted files that is being transferred.
2. External agents that seeks to make the encrypted image received non-decryptable by introducing certain information into the transferred encrypted file.
3. Finally, this system must be highly efficient in it's computational time consumed so that it take less time to encrypt and decrypt.

The objective of this paper is to implement one such system that satisfies such requirements.

# Proposed solution

- In this paper the proposed solution involves the use of a bit replacement DNA based chaotic-system based encryption and decryption system.
- Bit replacement technique.
- Chaotic system encryption.
- DNA based encryption.
- Why does this proposed solution works?
  - 1. this shatters the byte representation of the image file by taking the file and splitting it into two using BRT and then encrypting it.
  - 2. the DNA based encryption is highly non-linear
  - 3. the chaotic system has large set of keys which will protect against brute force attack to guess the secrete keys and it also generates numbers that are highly chaotic that cannot be reconstructed easily without knowing the initial keys.

# Background and prerequisites

Here is a quick background on the chaotic systems and DNA coding techniques , in manner which they are important in this discussion.

## *Chaotic systems:-*

### **What are chaotic systems and why are they used?**

Systems that generate a highly chaotic data(unpredictable)

Sensitive to the initial input parameters , so a little change in the input parameters results in vastly different outputs from these systems.

Thus, this is really useful as they can be used to generate a stream of highly unpredictable values that is sensitive to the initial condition but completely reproducible once the initial conditions are known thus this initial condition acts as the key which is used to get the very chaotic and unpredictable specific stream of data back during decryption.

So, this output of the system which is highly chaotic is used for the encryption and decryption processes.

Higher dimensional and hyper chaotic systems do better then lower dimensional and non-hyper chaotic systems.

### **The two chaotic systems used**

1. 3D Chen chaotic system.
2. 4D Lorenz hyper chaotic systems are used.

# 3D Chen chaotic system

Three-dimensional Chen chaotic system which was first introduced in 1999 by G. Chen can be modeled as:

$$\begin{aligned}\dot{x} &= a(y - x) \\ \dot{y} &= (c - a)x - xz + cy \\ \dot{z} &= xy - bz\end{aligned}$$

where  $a$ ,  $b$ ,  $c$  indicate to the Chen system parameters. In order to be in chaotic state, these parameters should have the following values:

$$a = 35, b = 3, c \in (20, 28.4)$$

Here this Chen chaotic system is used to encrypt in the confusion phase during the spatial rearrangement of the pixels positions.

# 4D hyper-chaotic Lorenz system

Four-dimensional hyper-chaotic Lorenz system can be expressed as follows:

$$\dot{x} = a(y - x)$$

$$\dot{y} = cx - xz - y + ew$$

$$\dot{z} = xy - bz$$

$$\dot{w} = -dy$$

were  $a, b, c, d, e$  point to the system parameters. 4D Lorenz hyper system is considered chaotic and can generate chaotic streams when the values of these parameters are:

$$a = 10, b = \frac{8}{3}, c = 46, d = 2, e = 12.$$

Here this Lorenz system is used in the encryption in the diffusion phase.

# DNA encoding techniques and DNA based operations

- Human DNA has 4 distinct nucleotides denoted by the letters A G T C
- Here A and T and complements and G and C are complements
- So, if they are represented using binary representation there are in total 24 ways to do this but if this complement nature is satisfied then there are only 8 total manners of representing this DNA nucleotides in binary.
- Thus, there are 8 rules for the representation of the DNA nucleotides using two bits for each nucleotides.

| DNA bases | Rule 1 | Rule 2 | Rule 3 | Rule 4 | Rule 5 | Rule 6 | Rule 7 | Rule 8 |
|-----------|--------|--------|--------|--------|--------|--------|--------|--------|
| A         | 00     | 00     | 01     | 01     | 10     | 10     | 11     | 11     |
| C         | 01     | 10     | 00     | 11     | 00     | 11     | 01     | 10     |
| G         | 10     | 01     | 11     | 00     | 11     | 00     | 10     | 01     |
| T         | 11     | 11     | 10     | 10     | 01     | 01     | 00     | 00     |

Fig1. rules for the encoding of the DNA into a two-bit binary number representation

- In this paper rule 2 in the above table is used or the encoding
- The rule of the encoding is that for every two bits that we take we look up from this table rule 2 and then replace it with the corresponding nucleotide symbol
- For decoding the DNA encoding we just take the nucleotide symbol and then replace it with the corresponding binary representation in the table rule 2



# DNA based addition and subtraction

In this paper the rule for the addition and the subtraction are given in the following table

| + | A | G | C | T | – | A | G | C | T |
|---|---|---|---|---|---|---|---|---|---|
| A | A | G | C | T | A | A | T | C | G |
| G | G | C | T | A | G | G | A | T | C |
| C | C | T | A | G | C | C | G | A | T |
| T | T | A | G | C | T | T | C | G | A |

Fig2. rules for the addition and subtraction of DNA nucleotides.

Overall DNA encoding and DNA based operations are used as it is a non-linear operation and which is hard for attackers to decrypt the information.

# *The algorithm and flowchart of the encryption algorithm*

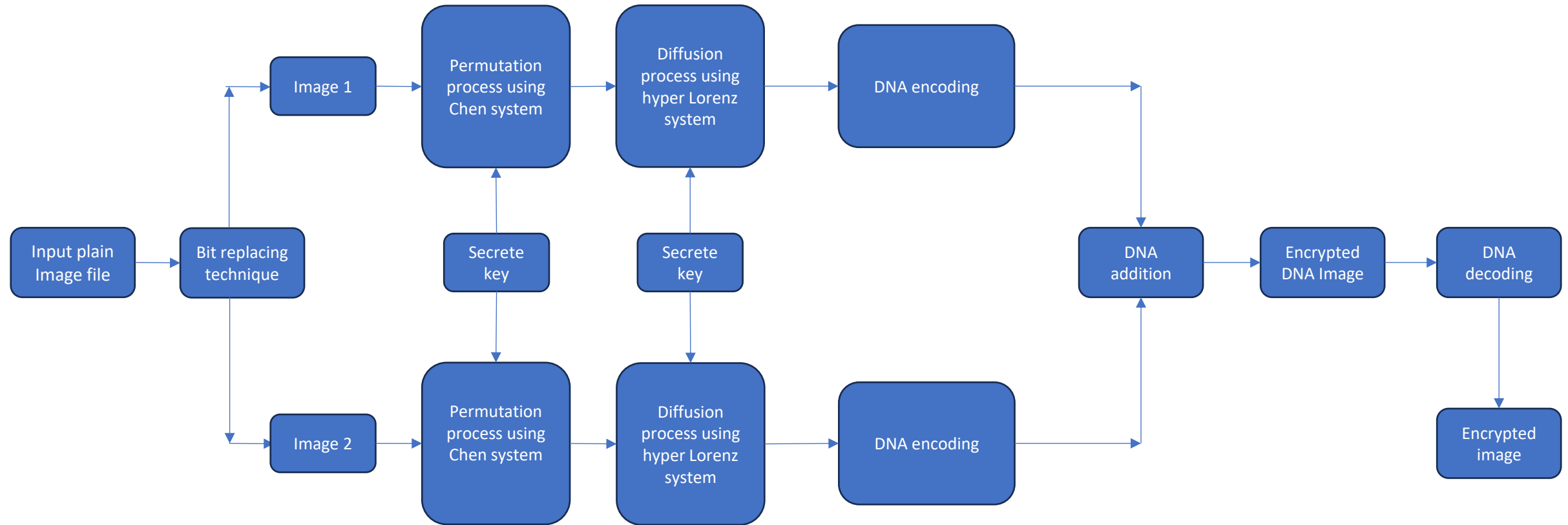


Fig3. Encryption algorithm block diagram/flowchart

# Bit level manipulation of the image file

## 1. Conversion of Image file to Grey Scale image file(if required for the processing of image into grey-scale)

The image file has four channels for each pixels that namely the

R G B A channels , each of values from 0 to 255 to represent the intensity of the each of the channels.

(it is to be noted that all of these number from 0 to 255 can be represented using 8 bits as  $2^8 = 256$ )

This image file with color is then converted to greyscale image where the R G B values are calculated using  $L = R * r + B * b + G * g$  , then

Setting the values of  $R = G = B = L$  in the new image file this new image file is the greyscale image.

Then for each pixel in this greyscale image the value of  $R = G = B = L$  , keeping the alpha channel A the same as before

## *2.Bit replacing technique to get two image files from one image file*

In this bit replacing technique, we take an image file and then apply the following method to get two image file outputs.

1. We take each of the pixel values and then convert it to its binary representation
2. We take that binary representation (which is going to be an 8-bit binary number) each of the
  1. 0's is replaced by 10
  2. 1's is replaced by 01 (to get a 16-bit binary number from each of the 8-bit binary number).
3. Then we take the
  1. first 8-bits in this 16-bit number and make this the value of the first image file .
  2. the next 8-bits as the value of the second image file.
4. This binary number is then converted back to the decimal value to get two image files.

And this is done for all the R G B values ,

So, at the end there are two image files.

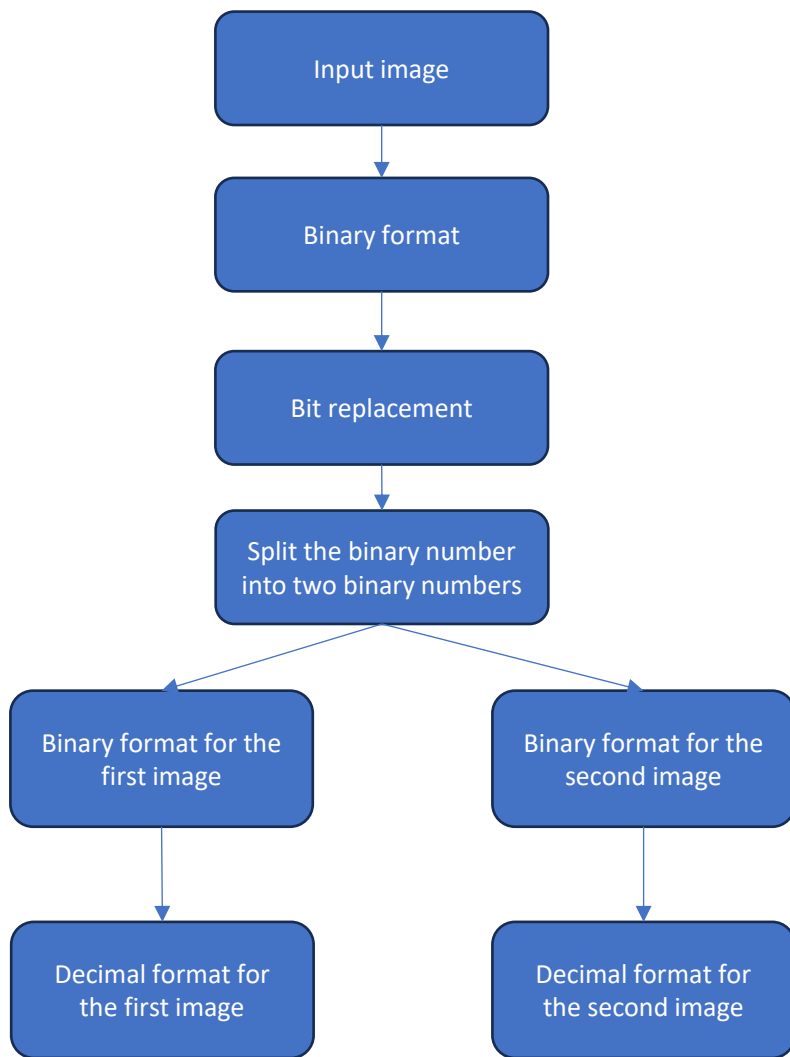


Fig4. Block diagram of the bit replacing technique used.

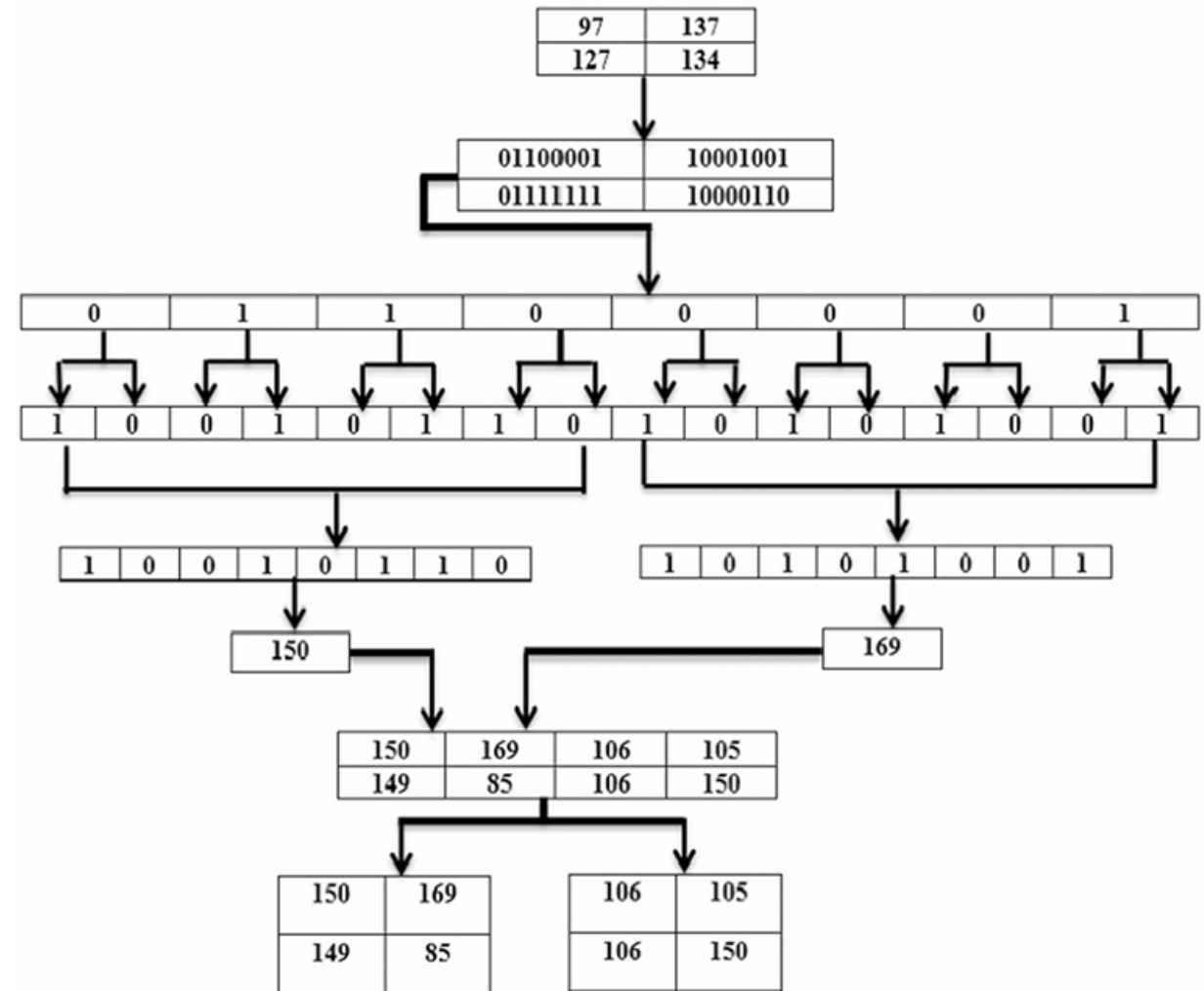


Fig5. Block diagram of the bit replacing technique used.

## *Stage 2 Chaos based encryption*

In this chaos-based encryption method we are to use the chaotic number generated by the chaotic algorithm to

- 1) Map the permutation of the pixels in the confusion phase and
- 2) The changing of the values of the pixels in the diffusion phase.

Assume that the image dimension is  $M \times N$

Here there are two chaos algorithms that takes in a single starting set of keys that acts as the initial condition for the chaotic system, then it uses this initial key or seed to generate  $M \times N$  number of chaotic numbers which are floating point values.

For each pixel this array of floating-point numbers is used to calculate the new position and the new pixel value for all the pixels in both of the image file.

# Chen chaotic system is used for the permutation of the pixel position

- 1.Chen chaotic system is used to generate  $M \times N$  number of chaotic floating-point values that are then used to map the new positions of the pixels in the original image.
- 2.Chen chaotic system has 3 chaotic variables  $x$  ,  $y$  and  $z$  and any one of them could be used for this purpose , let us select  $x$  and take  $M \times N$  number of chaotic floating-point values generated by  $x$  in the array  $X_1$  .
- 3.But this array  $X_1$  has floating point values , we need it in the integral value for our algorithm so we convert this set of floating-point values into integers within the range of 0 to 255 , for each of the values in this array get a new array after applying the following formula.  
That is get the new integer array  $X_2 = \text{mod}(\text{floor}(X_1 \times 10^{14}), 256)$ .
4. Array  $X_2$  is sorted then the mapping that is obtained in terms of the previous index  $i$  and the new index  $j \forall M \times N$  elements as mapping  $i$  to  $j$ .
- 5.That is element in index  $i$  in unsorted array is at index  $j$  in sorted array then  $i$  to  $j$  is now a mapping.
- 6.So, the  $i^{\text{th}}$  pixel will be placed in the  $j^{\text{th}}$  position  $i \in M \times N$

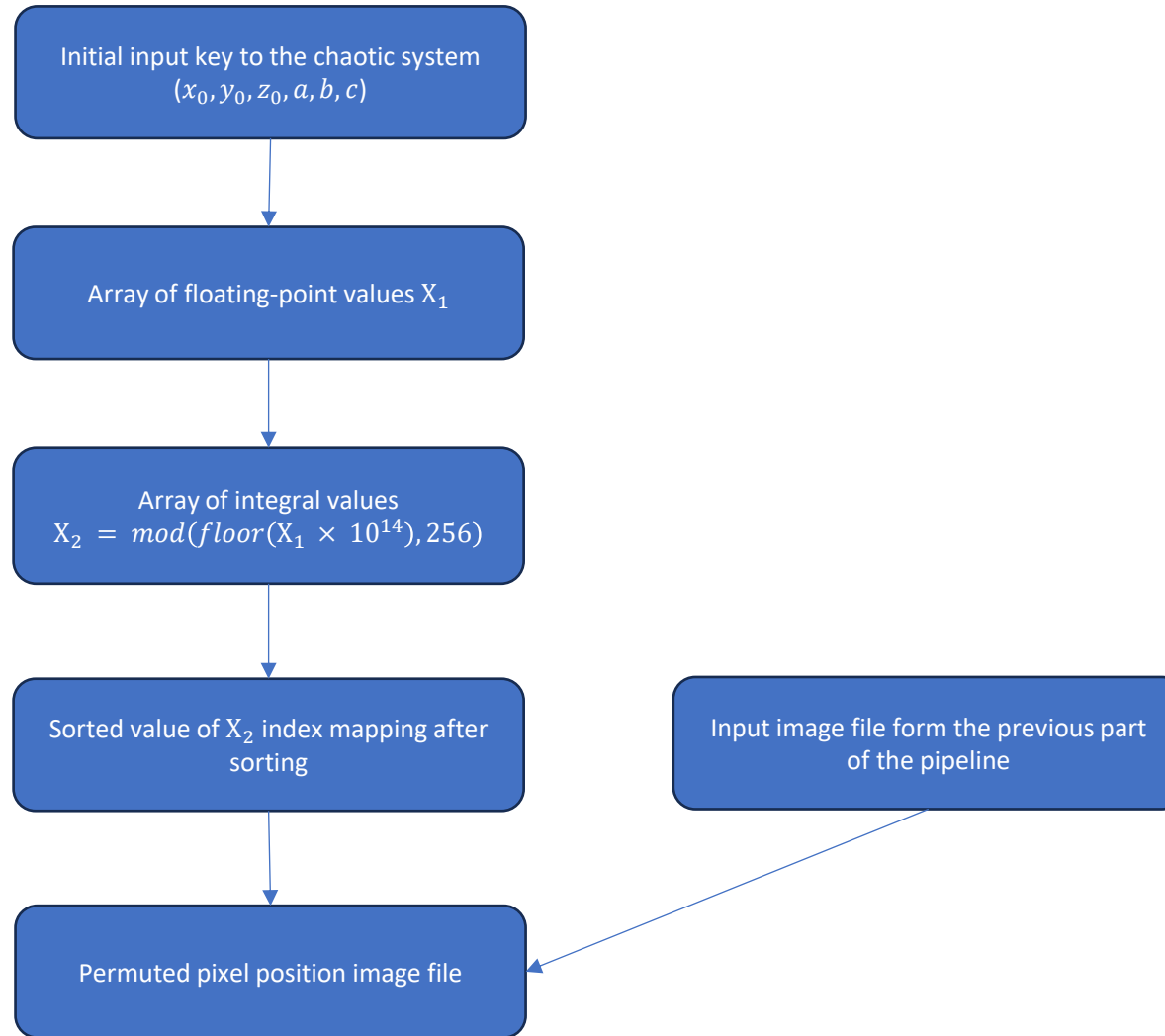


Fig6. Application of the Chen chaotic system to permute the pixel positions(block diagram).



# Lorenz's system used for the diffusion

$$\dot{x} = a(y - x)$$

$$\dot{y} = cx - xz - y + ew$$

$$\dot{z} = xy - bz$$

$$\dot{w} = -dy$$

This 4d hyper chaotic system has 4 chaotic parameters that could be used for the generation of  $M \times N$  chaotic numbers that will be then used to change the pixel value of the image. Any one of these chaotic parameters can be taken here we take  $y$  as the chaotic parameter.

1. It takes in the initial input key which is the seed for this and then runs the systems for some fixed number of iterations to generate  $M \times N$  number of chaotic floating-point values.
2. This array of  $M \times N$  floating point values are taken in the array  $Y_1$ .
3. New array of integral type values are generated from  $Y_1$  using the following equation for each element in  $Y_1$  to get the new array  $Y_2 = \text{mod}(\text{floor}(Y_1 \times 10^{14}), 256)$ .
4. Reshape  $Y_2$  to  $Y_3$  which is of the dimension of  $M \times N$ .
5. Then a Chained-XOR-operation is applied to apply the diffusion in this and get the new pixel values using the formula  $D(i, j) = D(i, j - 1) \oplus Y_3(i, j) \oplus D(i, j)$

Where  $D(i, j)$  is the pixel value of the pixel in the position  $i$  in the  $x$  axis and  $j$  in the  $y$  axis.

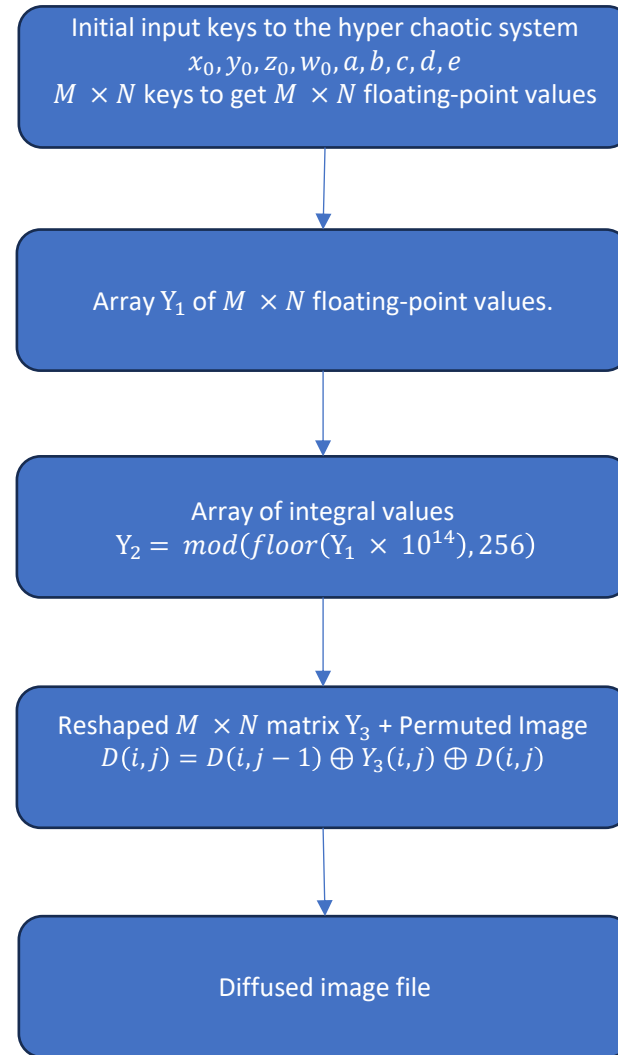


Fig7. Application of the Lorenz Hyper chaotic system to diffuse the pixel values(block diagram).

## *Stage 3 DNA encoding*

- Binary number of 8 bits is encoded using the replacement of two consecutive bits by the corresponding DNA nucleotide from the lookup table's rule 2.
- Then the DNA encoded image files are then taken together and then merged into one image file using the DNA addition rule or some reversible DNA based operation.
- And during the decoding stage this process is then reversed using the DNA subtraction rule or the reverse of the operation that was performed in the encryption phase.
- Then this encoded DNA sequence is then decoded using the lookup table into the binary format by replacing the DNA code by its binary representation from the lookup table's rule 2.
- Finally, this image obtained after the DNA decoding is then sent over the internet to the receiver.

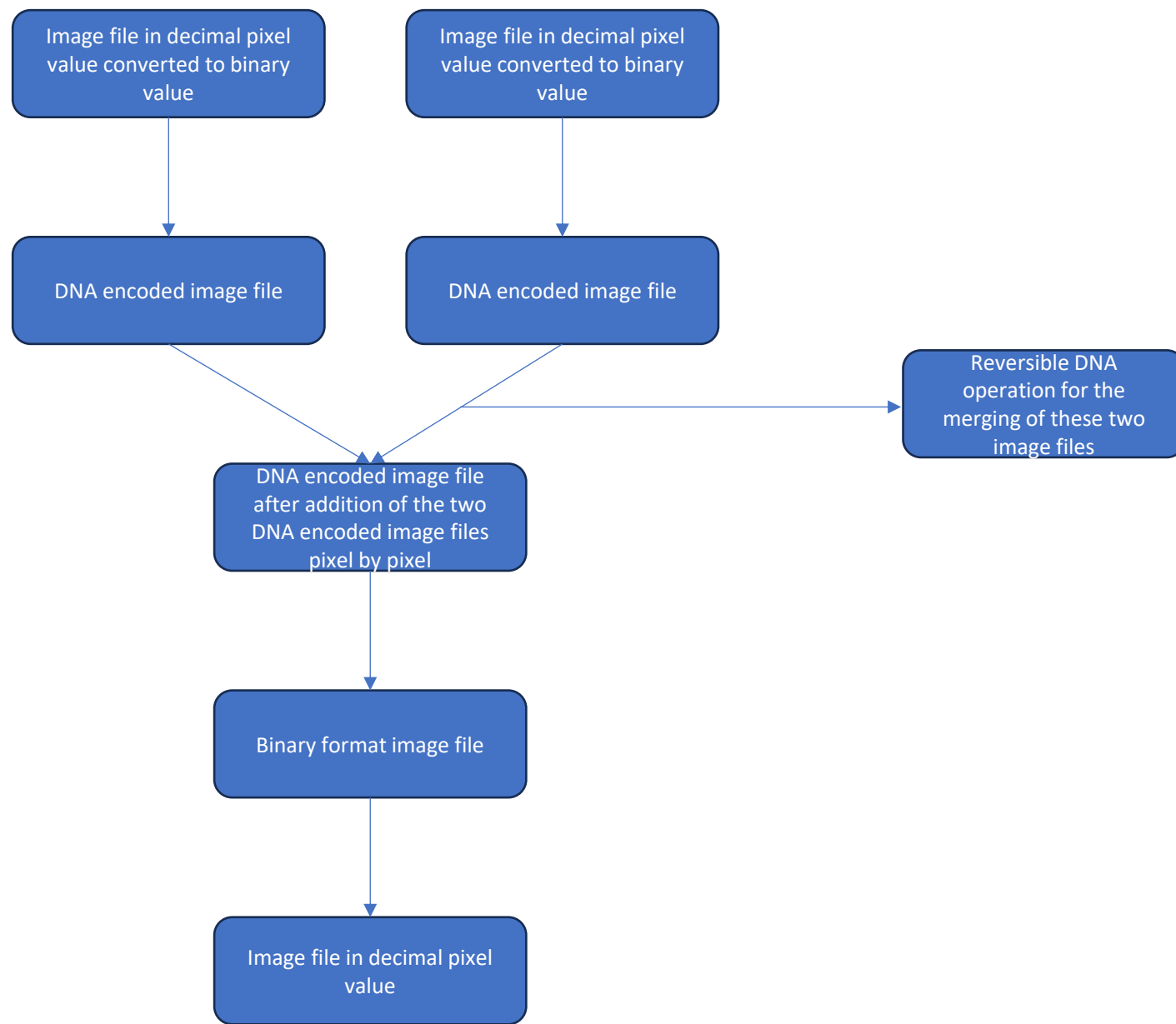


Fig8. DNA based encryption part of the image encryption process.

# Decryption block diagram

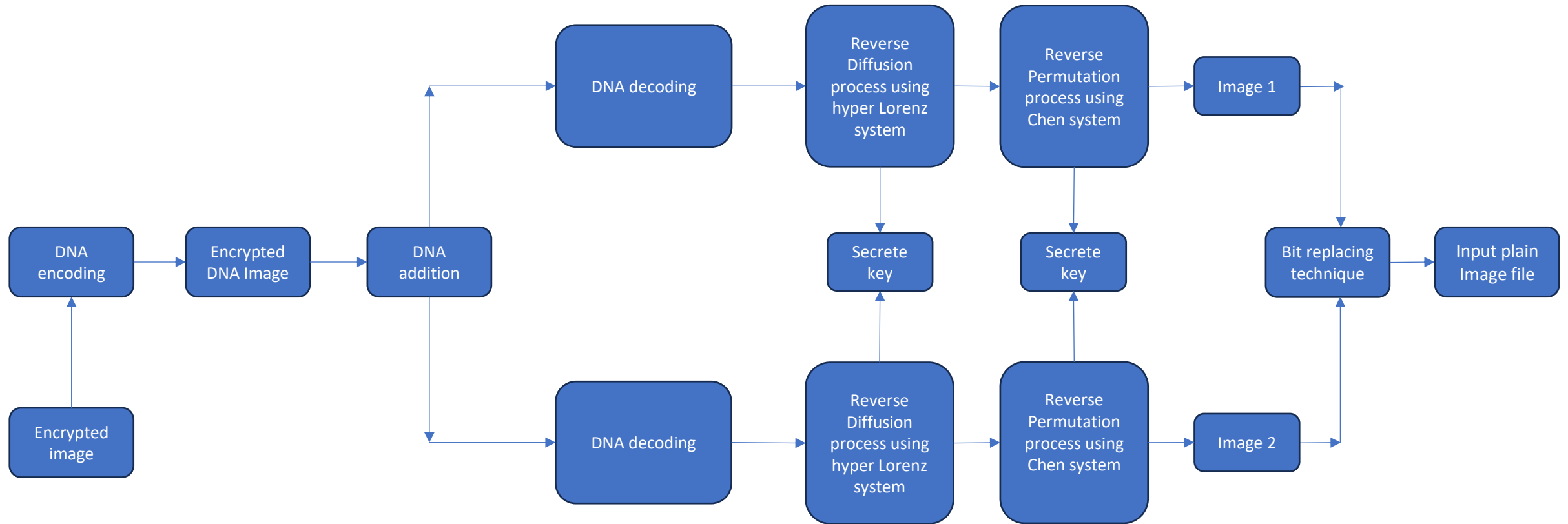


Fig9. Decryption algorithm block diagram/flowchart

# *Experimental simulation and analysis done.*

There are several analysis done to examine the performance and the effectiveness of the algorithm suggested

Analysis can could be broadly classified into the following categories

1. Analysis of the attack by external agent that seeks to find out information about the encrypted file that is being sent.
2. Analysis of the encryption algorithm from the encrypted files to find a decryption algorithm.
3. Analysis of the computational performance of the encryption and decryption algorithm.
4. Analysis of the algorithm against exterior attacks that introduces additional information which aims to make the decryption process fail.
5. Brute force attacks seeking to decrypt the algorithm.
6. And other miscellaneous attacks.

# Spatial analysis

- Check the spatial distribution of the pixels according to their values.
- The ideal case for the encrypted image should be that there is a uniform distribution of pixels of some value.
- That is there should not be an accumulation of pixels of certain values in one a region in the image file.
- This is overcame using in the confusion phase that reassigns the pixels position such that the resulting image does not have any more concentration of pixels of the same value in some region. This is measured using the

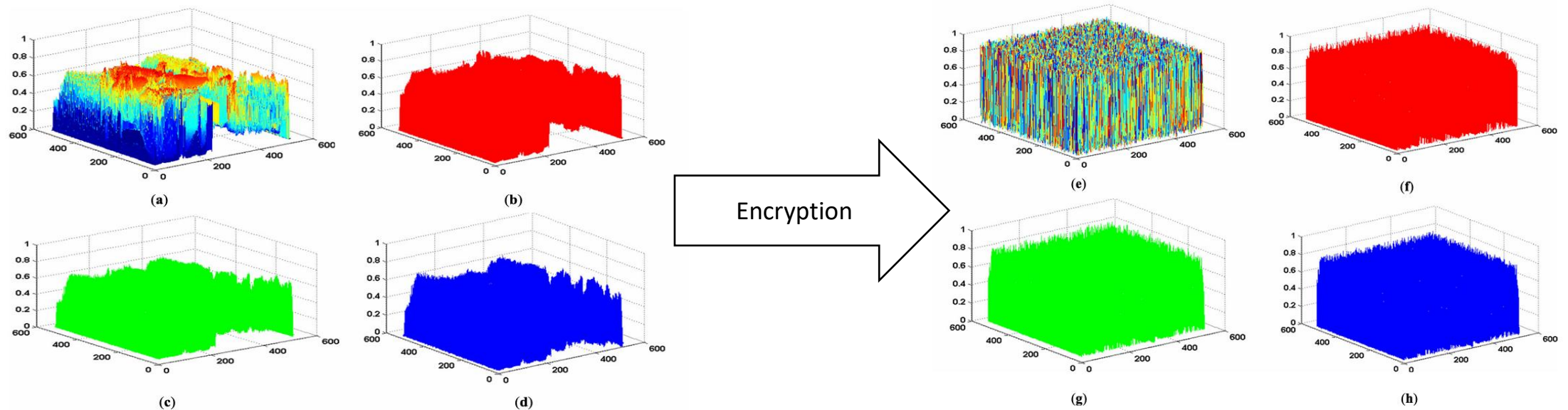


Fig. 10 Spatial distribution analysis in the **a** RGB channel, **b** Red, **c** Green, **d** Blue channels of the plain Barbara image, **e** RGB channel, **f** Red, **g** Green, **h** Blue channels of the encrypted Barbara image

# Histogram analysis

- Here even after the confusion phase there is still the same frequency distribution of the pixel of the same value in the original image and the scrambled confused imaged.
- This reveals information about the pixel value frequency distribution of the original image which the attacker can use.
- This is handled using the diffusion of the pixel values.
- After the diffusion of the pixel values, we aim to make the histogram a flat uniform frequency distribution of all the pixel values.
- Measurement of the uniformity of the histogram is done using the
  - Chi-Square Test ( $\chi^2$ )
  - P-value measurement which should be more the value of  $\alpha$ .

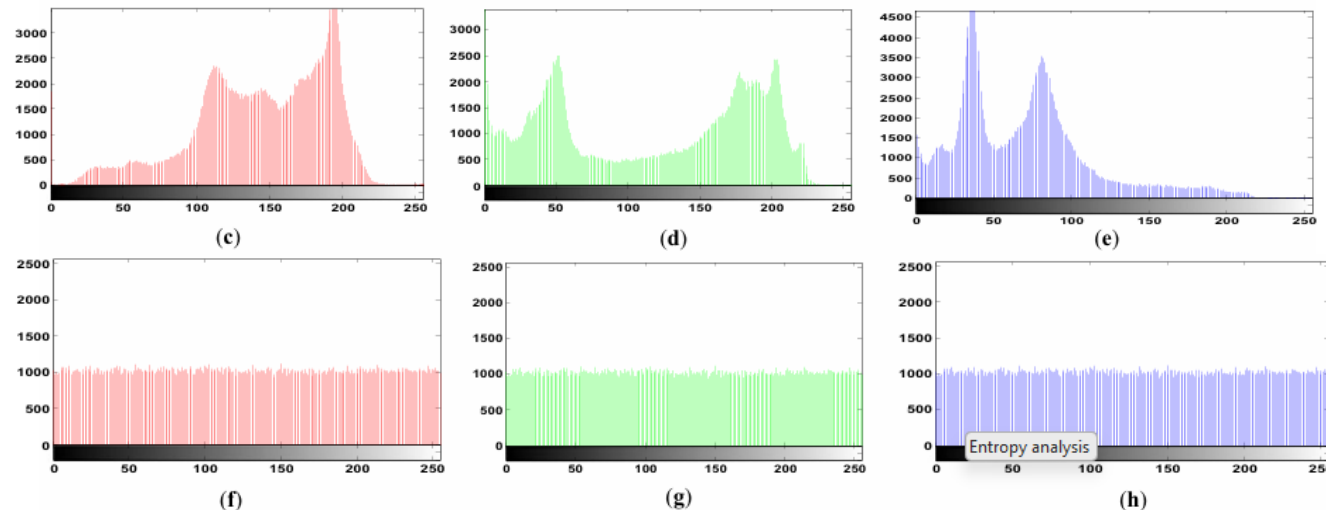


Fig11. Example of histogram analysis , for the R G B channels ; plaintext histogram(c, d, e) and encrypted image file(f , g, h)



- Chi-Square Test ( $X^2$ )
- **Concept:** It calculates the deviation between the observed pixel frequencies in the ciphertext and an idealistic, perfectly uniform distribution.
- **Mathematical Model:**
  - $$X^2 = \sum_{i=1}^{256} \frac{(o_i - e_i)^2}{e_i}$$
  - (Where  $o_i$  represents the observed pixel frequency,  $e_i$  represents the expected uniform frequency for the pixel  $i$ ).
- **The Ideal Case (Threshold):** For an 8-bit grayscale image (with  $d = 255$  degrees of freedom and a significance level of  $\alpha = 0.05$ ), a secure ciphertext must yield an  $X^2$  value strictly lower than **293.25**.
- **Observed Performance:** The proposed algorithm consistently scores between **235** and **290** across test images, definitively passing the uniformity requirement.
- Color Frequency Test (p-value)
- **Concept:** Derived from the Chi-square statistic, this measures the statistical probability that the image's pixel distribution is purely uniform.
- **Mathematical Model:**
  - $$p = \frac{1}{\frac{k}{2^2} \Gamma(\frac{v}{2})} \int_0^{X^2} t^{\frac{v}{2}-1} e^{-\frac{t}{2}} dt$$
  - (Where  $v$  refers to the degrees of freedom,  $v = k - 1$ ).
- **The Ideal Case (Threshold):** A highly structured plaintext image should have a probability ( $p$ ) near **0**. An ideally encrypted, noise-like image should have a probability ( $p$ ) approaching **1**.
- **Observed Performance:** In testing, plaintext images scored exactly **0**, while the algorithm's encrypted outputs scored overwhelmingly close to 1 (e.g., **0.995** and **0.959**), mathematically verifying complete structural diffusion.

# Entropy analysis

- Entropy here measures the extent of randomness in the distribution of the pixel values in the encrypted image file higher the entropy it means that there is very little information that can be inferred from this encrypted image by asking some form of question.
- Entropy analysis is divided into two parts the global entropy analysis and the local entropy analysis of the image.
- Local entropy measures the entropy in a selected region of the image.
- Global entropy measures the entropy for the whole image file.
- Entropy of the image is the probability distribution of all the values of the pixel in some region of the image.
- Thus, higher the entropy then the number of questions that needs to asked to know about the pixel value is maximized.
- So, for this example the entropy is ideally 8 as we need 8 questions to know the pixel value for a certain pixel , we ask what the pixel value is for all the 8 pixels

# Measurement of the entropy of the system

- Using Shannon's information theory, the amount of information that is needed **to represent the predictability of a data source is quantified in bits, measuring the fundamental uncertainty of the system.**
- And the entropy of the system is defined as the expected value of **the self-information across the probability distribution of all pixel intensities, indicating how much "randomness" exists.**
- Hence, the global entropy is (for this 8-bit information per pixel maximum entropy value is 8 , and the proposed algorithm cyphertext has entropy near 7.9988
  - $H(X) = - \sum_{i=0}^n P(x_i) \log_2 P(x_i)$
- And the local entropy is defined as (averaged across  $k$  randomly selected, non-overlapping blocks of pixels in the image file)
  - $\overline{H_{(k,T_B)}(S)} = \sum_{i=1}^k \frac{H(S_i)}{k}$

# Correlation analysis

Correlation analysis to know how much of a correlation or similarity there is for some randomly selected pixels with their adjacent pixels (that is pixels that are right next to it) , either in the horizontal vertical or the diagonal directions

As there is high correlation of the pixels and its neighbors in the plaintext image file this is not ideal as this reveals some information about the image to the attacker

The best way of hiding this is to make the correlation coefficient  $r_{xy}$  0 in the encrypted image file.

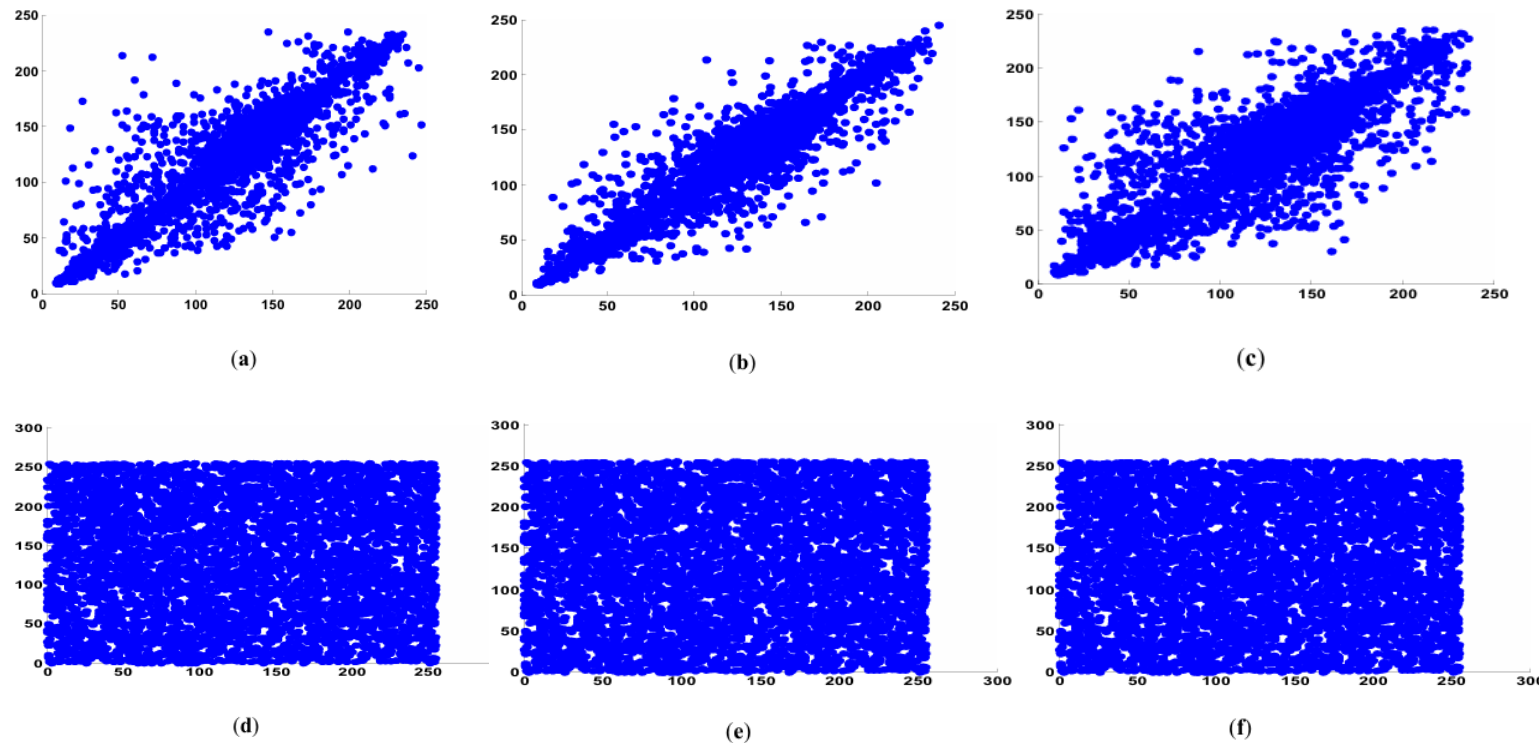
- **Expectation and Variance:** first the expected value  $E(x)$  and variance  $D(x)$  of selected pixel sequences of N pixels are calculated as:
  - $E(x) = \frac{1}{N} \sum_{i=1}^N x_i$
  - $D(x) = \frac{1}{N} \sum_{i=1}^N (x_i - E(x))^2$
- **Covariance and Correlation Coefficient:** The relationship between any two adjacent pixels (x and y) in horizontal, vertical, or diagonal orientations is given by
  - $cov(x, y) = \frac{1}{N} \sum_{i=1}^N (x_i - E(x))(y_i - E(y))$
  - $r_{xy} = \frac{cov(x, y)}{\sqrt{D(x)} \times \sqrt{D(y)}}$

**Testing Methodology:** 5000 pairs of adjacent pixels were randomly selected from both the original and encrypted images across horizontal, vertical, and diagonal orientations.

**Plaintext Vulnerability:** As expected, the original images exhibited extreme spatial correlation, with  $r_{xy}$  values approaching 0.98 to 0.99.

**Ciphertext Robustness:** Following the Chen permutation and Lorenz diffusion phases, the correlation coefficients dropped to absolute zero margins (e.g., 0.0007 or -0.0027).

**Conclusion:** The hybrid architecture successfully eradicates all structural patterns, rendering statistical correlation attacks mathematically impossible.



**Fig. 13** Correlation Analysis in **a** Horizontal, **b** Vertical, **c** Diagonal directions of plain Boat image, **d** Horizontal, **e** Vertical, **f** Diagonal directions of encrypted Boat image

# Signal-to-Noise Metrics

- **Objective:** To mathematically prove that the encrypted image (Y) retains zero visual similarity to the original image (X) and the decrypted image has similarity to the original image, using standard discrepancy metrics.
- **Mean Square Error (MSE):** Measures the average squared difference between the original and encrypted pixels. An ideal cryptosystem maximizes this value.

- $$MSE = \frac{1}{M \times N} \sum_{i=1}^M \sum_{j=1}^N [X(i, j) - Y(i, j)]^2$$

- **Peak Signal-to-Noise Ratio (PSNR):** Evaluates the noise level introduced by the encryption. Lower values indicate higher security.

- $$PSNR = 10 \times \log_{10} \left[ \frac{255 \times 255}{MSE} \right]$$

- **Structural Similarity Index (SSIM):** Compares the luminance, contrast, and structure of the two images structures that the human eye can see.

- $$SSIM(I_1, I_2) = \frac{(2\mu_{I_1}\mu_{I_2} + \alpha)(2\sigma_{I_1I_2} + \beta)}{(\mu_{I_1}^2 + \mu_{I_2}^2 + \alpha)(\sigma_{I_1}^2 + \sigma_{I_2}^2 + \beta)}$$

- **Experimental Results:**
- **MSE:** Ranges between 7234 and 14450 across samples, indicating catastrophic divergence from the plaintext.
- **PSNR:** Averages around 8 to 9 dB (a standard unencrypted transmission requires > 30 dB for clarity).
- **SSIM:** Scores average 0.03 (where 1.0 is a perfect match and 0.0 is completely uncorrelated).
- **Conclusion:** The ciphertext is mathematically proven to be pure noise relative to the input file, confirming high distortion efficiency.

# Validating Robustness Against Data Loss and Noise

- **The Threat Model:** During transmission over open channels, encrypted images may suffer from data loss (cropping attacks) or signal interference (noise attacks). A robust cryptosystem must allow the receiver to reconstruct a recognizable image even from damaged ciphertext.
- **Evaluation Methodology:** The encrypted image is subjected to severe data degradation, decrypted using the correct keys, and mathematically compared against the original plaintext.
- **Tested Attack Vectors:** The proposed architecture was tested against:
  - **Data Loss:** 12.5%, 25%, and 50% spatial cropping.
  - **Noise Injection:** Salt & Pepper, Gaussian, Speckle, and Poisson noise.
  - **Transformations:** JPEG Compression, Low-pass filtering, and Blurring.
- **Quantitative Recovery Metrics:**
  - **SSIM (Structural Similarity):** Reached up to **0.856** for 12.5% cropping and remained highly recognizable 0.339 even after 50% data loss.
  - **PSNR (Peak Signal-to-Noise Ratio):** Yielded recovery ratios between 10 dB and 20 dB across severe Gaussian and cropping attacks, confirming exceptional fault tolerance.
- **Conclusion:** The integration of the permutation-diffusion architecture naturally disperses local pixel data globally across the ciphertext. Consequently, localized damage to the encrypted file only causes faint, distributed noise in the decrypted file, rather than permanent structural destruction.

# Differential analysis

- **Objective:** To guarantee that identical images with only a 1-bit or 1-pixel difference produce entirely unrelated ciphertexts, to fight against differential cryptanalysis.
- **Metrics Used:**
- **NPCR (Number of Pixels Change Rate):** Measures the absolute number of pixels that change in the ciphertext when one plaintext pixel is altered. (Ideal:  $> 99.6\%$ ).
  - $NPCR = \frac{\sum_{i,j} D(i,j)}{M \times N} \times 100\%$ , where  $D(i) = \begin{cases} 0 & \text{if } C_1(i, j) = C_2(i, j) \\ 1 & \text{if } C_1(i, j) \neq C_2(i, j) \end{cases}$
- **UACI (Unified Average Changing Intensity):** Measures the average intensity of differences between the two ciphertexts. (Ideal: *approx* 33.4%).
  - $UACI = \frac{1}{M \times N} \sum_{i,j} \frac{|C_1(i,j) - C_2(i,j)|}{255} \times 100\%$
  - and represent the two distinct encrypted images (ciphertexts) produced after altering exactly one bit in the original input image
- **Performance:** The proposed algorithm achieves an average NPCR of **99.61%** and a UACI of **33.70%**, mathematically proving extreme sensitivity to plaintext alterations.



# Key Sensitivity Validation

- **Objective:** to show that near guess to the actual secret key used to decrypt will result in statistical noise
- **Testing Methodology:** The ciphertext was decrypted using the correct  $2^{747}$  bit key, and then decrypted again with a single initial parameter (like  $x_0$  in the Chen system) altered by a magnitude of  $10^{-15}$ .
- **Result:** Modifying the key by  $10^{-15}$  yields a decrypted image that is pure, unrecognizable static noise. The mathematical divergence is absolute, proving perfect key sensitivity. Measurements are done using the standard histogram analysis, correlation analysis and the entropy analysis.

# NIST Statistical Randomness Verification

- **Objective:** To verify that the pseudo-random key streams generated by the 3D Chen and 4D Lorenz systems are statistically indistinguishable from true white noise.
- **Testing Framework:** The generated bit sequences were subjected to the 15 rigorous tests of the National Institute of Standards and Technology (NIST) SP 800-22 test suite.
- **Performance:** The sequences successfully passed all 15 tests (including Block Frequency, Longest Run, and FFT tests) with p-values well above the 0.01 significance threshold, confirming the cryptographic safety of the chaotic generators.

# Key Space Security Analysis

## Exhausting Brute-Force Capabilities

- **Cryptographic Requirement:** To prevent exhaustive search (brute-force) attacks, a modern cryptosystem requires a minimum key space of  $2^{128}$ .
- **Algorithm Key Footprint:** This architecture derives its security from 15 distinct initial parameters and control values spanning both chaotic systems:
  - **3D Chen System:**  $x_0, y_0, z_0, a, b, c$
  - **4D Lorenz System:**  $x_0, y_0, z_0, w_0, a, b, c, d, e$
- **Calculation:** Assuming standard IEEE double-precision floating-point arithmetic with a computational precision of  $10^{-15}$ :
- **Conclusion:** A key space of  $2^{747}$  exceeds modern AES-256 standards, rendering brute-force cryptanalysis computationally infeasible given current computing architectures.

# Known / Chosen Plaintext Attack(CPA) Analysis

## Guaranteeing Zero Information Leakage

- **Threat Model:** In a Chosen Plaintext Attack (CPA), an adversary feeds specific, highly uniform data (like an all-black or all-white image) into the algorithm to observe how the cipher reacts and deduce the key stream.
- **Algorithm Responses:**
  - **All-Black Image Input:** Yields a completely chaotic, noise-like encrypted image.
  - **All-White Image Input:** Yields a completely chaotic, noise-like encrypted image.
- **Histogram Verification:** The histograms for both encrypted outputs remain perfectly flat and uniformly distributed.
- **Conclusion:** The integration of the bit-replacing technique and 4D Lorenz diffusion guarantees that the cipher's state heavily mutates regardless of the input entropy, perfectly neutralizing CPA vulnerabilities.

# Computational Complexity Analysis

## Hardware Efficiency and Algorithmic Big-O

- **Objective:** Robust security must not compromise real-time transmission speeds. The algorithm's runtime is analyzed using Big-O notation.
- **Phase Breakdown (for an image of size  $M \times N$ ):**
  - Bit Replacing Technique:  $O(M \times N)$
  - Chaotic Sequence Generation (Chen & Lorenz):  $2 \times O(M \times N)$
  - Permutation & Diffusion Matrix Operations:  $2 \times O(M \times N)$
  - DNA Encoding, Addition, and Decoding:  $2 \times O(M \times N)$
- **Total Complexity:**
  - Time Complexity =  $O(7 \times M \times N)$
- **Execution Time:** Evaluated at ~1.63 seconds for a  $512 \times 512$  grayscale image (Intel Core i3 environment).
- **Conclusion:** The linear time complexity relative to pixel count  $O(N)$  proves the architecture is highly optimized for fast, real-time multimedia streaming without sacrificing biological or chaotic complexity.

# Conclusion

## Why this proposed method is successful

- **Architectural Success:** The proposed scheme successfully bridges discrete bit-level manipulation, continuous high-dimensional chaos (Chen and Lorenz), and non-linear biological computing analogs (DNA Base-4 arithmetic).
- **Security Validation:** \* Eradicated spatial correlation ( $r_{xy} \approx 0$ ).
  - Maximized information entropy ( $H(x) \approx 7.999$ ).
  - Provided an impenetrable key space ( $2^{747}$ ).
- **Operational Viability:** The  $O(7MN)$  computational complexity ensures the system remains lightweight and viable for constrained embedded systems and active telecommunication networks.
- **Final Verdict:** The algorithm establishes a highly secure, attack-resistant and computationally efficient framework capable of protecting sensitive images across open channels.